Report: An AI Agent for Generative Modeling

Source code: `https://github.com/yanruoz/gen-model-agent` BIOS 774, Project 2

1. System architecture

The agent takes a dataset folder and a free-text objective and runs with one command: `./run.sh <dataset> "<objective>"`. `run.sh` creates an isolated Python environment (`venv/`), downloads the dataset's file if missing (checked against its declared MD5 checksum), and starts Claude Code in headless mode (`claude -p`) with the agent's instructions (`AGENT.md`) as an appended system prompt. The run loads no user or project settings. A permission file lets the agent run only `venv/bin/python scripts/*`, read the repository, and write under `outputs/`; anything else is denied without a prompt, so a run never stalls waiting for a person.

The design separates *deciding* from *computing*. The language model reads the data profile and five *skill* files (Markdown decision rules for objective interpretation, preprocessing, model selection, downstream use, and reporting) and writes its decisions as JSON. Every number and figure comes from deterministic, seeded Python scripts. Each stage reads and writes JSON under `outputs/<dataset>/`, so any stage can be rerun on its own:

1. **Profile** (`profiler.py`): size, modality (image or tabular), whether values are counts, sparsity, class balance.
2. **Interpret the objective** into `objective.json`: task, success criterion, and a decision: *proceed*, *proceed_adapted* (run a related task that can be tested on this data, and say so), or *decline*. Checked by `validate_objective.py`.
3. **Plan** (`plan.json`): held-out split, preprocessing, models, hyperparameters, and seeds, each with a dataset-specific reason. Checked by `validate_plan.py`.
4. **Execute** (`run_plan.py`): preprocess once; train, generate, and evaluate each model and seed; run the test for the objective (Section 2); draw figures.
5. **Critique**: a separate Claude instance (the `gen-critic` subagent) sees only the objective, plan, metrics, and training logs, not the main agent's reasoning. It checks whether any metric tests the success criterion, whether the reference values are sensible, whether seeds agree, whether training ran long enough, copying and privacy-claim risk, and whether results contradict the stated reasons. The agent then makes exactly one plan revision.
6. **Recommend** downstream uses in `recommendation.json`, checked by `validate_recommendation.py`.
7. **Report**: the agent writes `findings.md`; `report.py` builds the PDF from the JSON files.

Implemented models (PyTorch, small enough for a laptop): for tabular data, a variational autoencoder (VAE) with a negative-binomial (NB, for counts), Gaussian, or Bernoulli output, and a RealNVP normalizing flow; for images, a class-conditional convolutional VAE, a conditional generative adversarial network (GAN; projection discriminator, hinge loss), and a conditional diffusion model (small UNet, classifier-free guidance, DDIM sampler).

2. How the agent makes generative-modeling decisions

Whether to model. `validate_objective.py` enforces data checks in code. Too few samples means decline. A task that needs labels on unlabeled data, or a task with no implemented test on this modality (for example uncertainty quantification), must be declined or adapted, so the agent never trains models whose output nobody can judge. Near-balanced classes (largest/smallest class ratio below 1.5) cannot be treated as an imbalance problem. For class balancing, a **headroom check** first trains the same classifier (three seeds) on real data, on real data with the rare class resampled, and on real data plus *new real* images of the rare class left out of the training subsample. New real data is the best any generator could do. If it does not beat resampling, the plan trains a single generator only to confirm this, and the recommendation expects resampling to be enough.

Which model. Each family has a fixed capability profile (exact likelihood, latent space, conditional generation, reconstruction), and `validate_plan.py` rejects a model that lacks what the task needs, for example a GAN for anomaly detection, since it has no likelihood. The model-selection skill picks *one* primary model from the objective and data (an NB VAE on raw counts for single-cell data, conditional diffusion for labeled images) and allows at most one comparator, with a stated reason. The aim is to serve the objective, not to rank every model.

Hyperparameters come from heuristics keyed to the profile: latent size by sample count, epochs as a maximum with early stopping on a validation slice of the training data, and per-epoch timings to fit the time budget. Every value needs its own reason; the validator rejects placeholders such as "default".

Evaluation is fixed per modality, not chosen by the agent, and always uses held-out real data. Tabular data is compared in a principal-component (PCA) space limited to components that generalize to held-out data; images are compared in the 128-dimensional features of a small convolutional neural network (CNN) classifier trained on the real images. Metrics: *C2ST* (classifier two-sample test: the area under the ROC curve, AUC or AUROC, of

a classifier separating synthetic from real samples; 0.5 means indistinguishable); *MMD*^{2} (squared maximum mean discrepancy, a kernel distance between distributions; 0 means equal); *precision/recall* (share of synthetic samples that fall near real ones / share of real samples covered by synthetic ones); *Fréchet distance* (the Fréchet Inception Distance, FID, formula, computed on the classifier's features); *label agreement* (share of generated images that the real-trained classifier assigns to the requested class); and *DCR ratio* (median distance from synthetic samples to their closest training record, divided by the same for held-out real samples; below 1 suggests copying). Each metric has a *reference*: the value two real samples of the same size give, i.e. what a perfect generator would score.

Then comes the objective test, always against a non-generative baseline: for representation learning, how well the latent space keeps each cell's 15 nearest neighbors (*trustworthiness*, 0 to 1) compared with PCA of the same size; for replacing real data, *train on synthetic, test on real* (TSTR): the same classifier trained on n synthetic or n real images and scored on held-out real images, whose ratio is the *utility ratio* (1 means synthetic is as useful as real); for class balancing, rare-class F1 with synthetic images vs with resampling; for anomaly detection, the AUROC for separating corrupted held-out samples from normal ones, against k-nearest-neighbor (kNN) distance and PCA reconstruction error; for imputation, error on deliberately hidden held-out values, against mean and kNN imputation.

Downstream use. Each recommended use must cite a metric. The validator checks the cited path and value against the metrics files, rejects privacy claims ("anonymized", "safe to share"), and requires a statement that no formal privacy guarantee is given whenever sharing comes up.

3. Tools and AI models used

Claude Code (headless `claude -p`, Claude Opus) is the only AI model involved. Models and metrics use PyTorch 2.14, NumPy, SciPy, and scikit-learn; figures and reports use Matplotlib. The metric choice follows common practice for synthetic data; the synthcity benchmark (Qian et al., NeurIPS 2023) was the reference for tabular fidelity and privacy metrics. VAE samples are drawn from a 10-component Gaussian mixture fit to the training latent codes, instead of the standard normal prior (ex-post density estimation, Ghosh et al., ICLR 2020).

4. Results

These numbers come from rehearsal runs of the final pipeline, each one `./run.sh` command on an empty `outputs/`. Generated reports 1 and 2 come from separate clean runs of the same commands; because the agent's decisions can vary between runs, their numbers may differ slightly.

PBMC 3k (2,700 cells × 32,738 genes, 97.4% zeros, raw counts, no labels). Objective: "Learn a low-dimensional representation and generate realistic synthetic cells for sharing." The agent read this as representation learning plus data generation. It kept the 2,000 most variable genes, held out 20% of cells, and chose an NB VAE on raw counts (latent size 10, two seeds) as the primary model, with a RealNVP flow on 20 principal components as the comparator. The critic found that the flow overfit by epoch 13 and had one seed; the revision made the flow smaller, lowered its learning rate, and added a second seed. The VAE runs were reused.

	VAE seed 0	VAE seed 1	flow seed 0	flow seed 1
C2ST AUC (reference 0.58 / 0.56)	0.865	0.842	0.855	0.809
MMD^2 (reference 0.0003)	0.0075	0.0053	0.0086	0.0054
precision / recall	0.94 / 0.74	0.94 / 0.80	0.58 / 0.98	0.58 / 0.98
DCR ratio	1.14	1.15	1.61	1.52
trustworthiness (10-component PCA: 0.833)	0.803	0.794	–	–

The VAE reproduces per-gene statistics (correlation with held-out real data: 0.998 for means, 0.993 for zero fractions; median library size within 1%), but a classifier still separates its cells from real ones (C2ST 0.84–0.87), and its latent space keeps neighbors slightly worse than PCA. DCR ratios above 1 show no sign of copying. The flow covers the real cells (recall 0.98), but 42% of its samples are not near any real cell. The agent reported that both parts of its success criterion failed, recommended neither the latent codes nor the synthetic cells as stand-ins for real cells, allowed the synthetic counts only as test data for software that needs realistic per-gene distributions, and stated that no formal privacy guarantee is given.

PathMNIST (89,996 $28 \times 28 \times 3$ images, 9 tissue classes, largest/smallest class ratio 1.63). Objective: "Generate realistic tissue images of each class that could replace the real data for training a tissue classifier." The agent trained conditional diffusion on a stratified 20,000-image subsample and ran TSTR with $n = 10{,}000$ and three classifier seeds per arm. The first model (110 epochs) reached a utility ratio of 0.66, and its validation loss was still falling

at the last epoch. The critic flagged this, and the revision retrained for 155 epochs.

	110 epochs	155 epochs (final)	reference
TSTR accuracy (real-trained: 0.913 ± 0.002)	–	0.763 ± 0.014	
utility ratio	0.66	0.84	1
Fréchet distance	26.5	21.9	0.29
C2ST AUC	0.982	0.972	0.506
label agreement	0.846	0.888	0.940

Longer training improved every metric, but the synthetic images cannot replace real ones: the classifier loses 15 accuracy points, and generated images of classes 6 and 7 are recognized as their class only 68% and 67% of the time. The agent recommended against replacement and allowed the images only as illustrative examples.

Robustness: other objectives on the same data. On PBMC, "Recover gene expression values lost to dropout" was read as imputation. The agent trained only the VAE (the only family the imputation test supports) and hid 10% of each held-out cell's nonzero counts. The VAE's RMSE on $\log(1 + \text{count})$ was 0.512, against 0.550 for the library-scaled mean and 0.633 for kNN. Its verdict was *conditional*: usable before exploratory analysis, but not evidence that any given zero is a dropout.

"Flag unusual cells that may be doublets or low-quality captures" was read as anomaly detection. The agent trained the NB VAE, scoring cells by their negative ELBO (evidence lower bound), and a flow on PCA components as comparator, two seeds each, and tested them on held-out cells mixed with simulated doublets (two cells' counts summed) and gene-shuffled cells. On doublets the VAE reached AUROC 0.935, against 0.708 for kNN distance. The flow (0.14) and PCA reconstruction (0.11) rated doublets as *more* typical than real cells; one untested explanation is that after normalization a doublet is an average of two cells and lies near the center of the data. Every method detected gene-shuffled cells (AUROC 0.995–1.0), so the VAE adds nothing there. The one positive verdict was conditional: use the VAE score to shortlist possible doublets for manual review, alongside the usual cutoffs on total counts and genes detected, because no real doublets or low-quality cells were tested.

During development, "Balance the rarest tissue class to improve a classifier" on PathMNIST ran the headroom check: accuracy was 0.912 on real data, 0.923 with resampling, and 0.923 with new real images of the rare class. At this mild imbalance even perfect synthetic images could not beat resampling, and adding diffusion images gave 0.909.

Why no GAN ran. The conditional GAN is implemented and tested, but the agent never chose it. The rules make conditional diffusion the primary model for labeled images, because it trains stably and follows the class label reliably. A GAN is the comparator only when the plan needs faster sampling and says why. The validator excludes GANs from anomaly detection, imputation, and representation learning, since a GAN has neither a likelihood nor an encoder, and the skill treats a GAN as the least stable choice for a few thousand tabular rows. A GAN also needs at least two seeds (about 8 minutes each here), because its results depend strongly on the seed. In the PathMNIST run the first diffusion model used 18.6 of the 30 minutes, leaving less than the 16 two GAN seeds need. The reports therefore contain no GAN-vs-diffusion comparison. This is a gap: a GAN might give sharper images, at the cost of covering fewer image types.

5. Strengths and limitations

Strengths.

- *It tests the user's goal, not just sample quality.* Each objective maps to a fixed test on held-out real data against a non-generative baseline, so negative results are reported as such. In every run above the verdicts matched the numbers, including "not recommended" for both main objectives.
- *Decisions are checked in code.* Validators reject a model that lacks the needed capability, vague reasons, budget overruns, uncited downstream claims, and privacy claims. The headroom check keeps the agent from promising gains that even new real data would not give.
- *The critique changed results.* On PathMNIST it raised the utility ratio from 0.66 to 0.84; on PBMC it fixed an overfit comparator.
- *Isolation and reproducibility.* One command, an isolated environment, seeded training, and a run that loads no user settings and can execute only the project's scripts.

Limitations.

- *Small models and budgets.* Models are sized for a laptop and a 30-minute training budget. Image Fréchet

distances use a small PathMNIST classifier, not the Inception network, so they are not comparable to published FIDs.

- *Few seeds, one split.* Most results use two or three seeds on one held-out split, so the reported spread covers training noise, not variation across splits or masks.
- *Limited task coverage.* Uncertainty quantification has no test and is always declined or adapted; class balancing is tested only on images; categorical tabular features are not supported.
- *No privacy guarantee.* The DCR ratio is evidence against copying, not differential privacy.
- *Run-to-run variation.* The agent's decisions are not deterministic, and a retrain after critique can nearly double the PathMNIST run time (about 65 minutes in the rehearsal).
- *Issue found in rehearsal.* One run lost write access after the agent ran `cd`, because path permissions were relative to the working directory. `run.sh` now passes absolute path rules and denies `cd`.